OPERATING SYSTEMS

Detailed Study + Interview Notes

Processes | Scheduling | Synchronization | Deadlocks | Memory

How to use these notes
Study the concepts first. Then use the interview-question pages for active recall. Each answer is intentionally structured around a precise definition, why it matters, a simple example, and the key distinction/trade-off to remember.

Roadmap scope

- Processes: process, thread, process vs thread, context switch
- CPU scheduling: FCFS, SJF, Priority, Round Robin
- Synchronization: race condition, critical section, mutex, semaphore
- Deadlocks: four conditions, prevention, avoidance, detection
- Memory: stack, heap, paging, virtual memory, page fault, fragmentation

Interview questions covered: 10 roadmap questions, including process vs thread, context switching, race conditions, mutex vs semaphore, deadlocks, virtual memory, page faults, and stack vs heap.

1. MASTER MAP

The goal is to see how the topics connect instead of memorizing isolated definitions.

Processes	A program in execution; managed by the OS
Threads	Units of execution inside a process
Scheduling	Decides which runnable process/thread gets CPU time
Synchronization	Controls access to shared state
Deadlocks	Permanent waiting caused by resource dependencies
Memory	Provides address spaces and manages physical/virtual memory

Core relationships
Process -> threads -> scheduling: a process may contain multiple threads, and the scheduler chooses runnable execution units.
Concurrency -> synchronization: when execution units share state, synchronization prevents incorrect interleavings.
Synchronization -> deadlock: poor locking/resource acquisition can create circular waiting.
Virtual memory -> paging -> page fault: virtual pages may be mapped to physical frames; if a needed page is not resident, a page fault occurs.

2. PROCESSES

2.1 Process

Concept: A process is a program in execution. It has its own execution context and resources managed by the operating system.

Think: program = passive code; process = active execution of that code.

Typical process context includes: program counter, CPU registers, address space, open resources and OS bookkeeping.

2.2 Thread

Concept: A thread is an execution unit within a process. Threads in the same process share the process's address space and many resources, while each thread has its own execution state such as a program counter and stack.

2.3 Process vs Thread

Aspect	Process	Thread
Address space	Own separate virtual address space	Shares process address space
Resources	Own process resources/context	Shares many process resources
Creation/switching	Generally heavier	Generally lighter
Communication	Often needs IPC	Shared memory is naturally available inside the process
Failure isolation	Stronger isolation	Failure can affect the whole process

2.4 Context Switch

Concept: A context switch occurs when the CPU stops running one execution context and starts another, requiring the OS to save the current context and restore the next one.

Typical flow
1. Save current execution state -> 2. Scheduler selects another runnable unit -> 3. Restore its saved state -> 4. Resume execution.

Why it costs time: saving/restoring registers and scheduler bookkeeping add overhead; CPU time spent switching is not useful application work.

3. CPU SCHEDULING

The scheduler chooses which ready process/thread should run on the CPU. Common criteria include waiting time, turnaround time, response time, throughput and fairness.

FCFS (First-Come, First-Served)

Runs jobs in arrival order. Simple and easy to implement, but a long job at the front can delay many shorter jobs.

SJF (Shortest Job First)

Chooses the job with the shortest CPU burst. It can minimize average waiting time when burst lengths are known/estimated, but long jobs may wait.

Priority Scheduling

Runs the highest-priority ready job. Priorities can be static or dynamic; low-priority jobs can starve unless techniques such as aging are used.

Round Robin

Each ready job receives a time quantum in cyclic order. It is designed for responsiveness and fairness in time-sharing systems; the quantum affects overhead and responsiveness.

Quick comparison

Algorithm	Main idea	Strength	Risk / trade-off
FCFS	Arrival order	Very simple	Convoy effect / poor response
SJF	Shortest burst first	Low average waiting in ideal conditions	Needs burst estimate; starvation possible
Priority	Highest priority first	Supports importance levels	Starvation possible; needs priority policy
Round Robin	Time quantum per job	Responsive and fair	Too-small quantum increases switching overhead

Study cue: FCFS asks "who arrived first?" SJF asks "who needs the least CPU burst?" Priority asks "who is most important?" Round Robin asks "who has waited longest for another time slice?"

4. SYNCHRONIZATION

When multiple threads/processes access shared state concurrently, the order of operations may matter. Synchronization provides mechanisms for controlling those interactions.

4.1 Race Condition

Concept: A race condition occurs when the program result depends on the timing/interleaving of concurrent operations on shared state.

Example: two threads execute `counter = counter + 1`. If both read the same old value before either writes back, one increment may be lost.

4.2 Critical Section

Concept: A critical section is the part of code that accesses shared data/resource and therefore must be executed under appropriate synchronization so conflicting concurrent execution does not corrupt the state.

4.3 Mutex

Concept: A mutex is a mutual-exclusion lock that allows only one thread at a time to own the protected critical section.

4.4 Semaphore

Concept: A semaphore is a synchronization primitive with a counter. Threads can wait/decrement and signal/increment it; a binary semaphore can act similarly to a lock for some designs, while counting semaphores are useful for limited resource pools.

5. DEADLOCKS

Deadlock: a set of processes/threads is permanently waiting because each holds or depends on resources needed by others in the set.

5.1 The Four Necessary Conditions

Condition	Meaning
Mutual exclusion	At least one resource is non-shareable: only one execution unit can use it at a time.
Hold and wait	An execution unit holds at least one resource while waiting for another.
No preemption	Resources cannot be forcibly taken away; they must be released voluntarily.
Circular wait	There is a cycle in which each execution unit waits for a resource held by the next.

5.2 Prevention

Prevention deliberately breaks at least one necessary condition. Examples: require resources to be requested together (break hold-and-wait), impose a global resource ordering (break circular wait), or make certain resources preemptible where feasible.

5.3 Avoidance

Avoidance does not reject a condition in advance. Instead, the system examines resource-allocation choices and grants requests only when the resulting state remains safe. The classic example is the Banker-style safety idea.

5.4 Detection

Detection allows deadlock to occur, then periodically or on demand checks for cycles/unsafe resource-allocation patterns. Once detected, the system needs a recovery strategy.

Memory hook: Prevention = make deadlock impossible. Avoidance = refuse unsafe choices. Detection = allow, then find and recover.

6. MEMORY

6.1 Stack

In memory management: the call stack stores call frames and automatic/local execution state. It is associated with function calls and generally grows/shrinks with execution.

6.2 Heap

In memory management: the heap is the region used for dynamic allocation managed by a runtime, allocator, or language memory manager. Objects/data can live beyond a single function call depending on allocation and lifetime rules.

6.3 Paging

Concept: Paging divides virtual memory into fixed-size pages and physical memory into fixed-size frames. A page can be mapped to a frame using page tables.

Virtual page -> page table mapping -> physical frame
The process uses virtual addresses; the MMU and OS-managed page tables translate them toward physical memory.

6.4 Virtual Memory

Concept: Virtual memory gives each process an address-space abstraction that is larger/more flexible than the set of physical frames currently assigned to it. The OS and hardware translate virtual addresses and can move inactive pages between memory and storage.

6.5 Page Fault

Concept: A page fault occurs when a process accesses a virtual page that is not currently mapped to an available physical frame in the required way. The OS handles the fault, potentially bringing the page from secondary storage, updating mappings, and resuming execution.

6.6 Fragmentation

Internal fragmentation: wasted space inside allocated blocks because allocation occurs in fixed or larger units than requested.
External fragmentation: free memory exists but is split into separated holes, making large contiguous allocations difficult.

7. INTERVIEW QUESTIONS - CANONICAL ANSWERS

These are the ten questions from the roadmap. Use them for active recall after studying the full topic sections.

1. Process vs thread?

Definition
A process is a program in execution with its own virtual address space and OS-managed resources. A thread is an execution unit inside a process and shares the process's address space and many resources with sibling threads.

Why / significance: The distinction matters because threads are generally lighter and communicate through shared memory more easily, while processes provide stronger isolation.

Example / recall: Example: a browser process may contain multiple threads for UI, networking and background work.

2. Why are threads cheaper?

Definition
Threads are generally cheaper because they share the process's address space and many resources, so creating/switching between threads usually requires less resource setup than creating/switching between separate processes.

Why / significance: Less isolation and shared state also mean synchronization is required when threads access common data.

Example / recall: Example: two worker threads in one process can share in-memory data without setting up process-to-process communication.

3. What is context switching?

Definition
A context switch is the OS operation of saving the current execution context and restoring another process/thread context so the CPU can resume a different execution unit.

Why / significance: It enables multitasking but introduces overhead because the CPU spends time switching rather than doing application work.

Example / recall: Typical steps: save registers/state -> choose next runnable unit -> restore state -> resume.

4. What is a race condition?

Definition
A race condition occurs when multiple concurrent execution units access shared state and the final result depends on their timing or interleaving.

Why / significance: It can produce incorrect or nondeterministic results. The usual remedy is to protect the relevant critical section with an appropriate synchronization mechanism.

Example / recall: Example: two threads increment the same counter without synchronization and one update is lost.

5. Mutex vs semaphore?

Definition
A mutex is primarily an ownership-based mutual-exclusion lock: one thread owns the lock while inside the protected region. A semaphore is a counter-based synchronization primitive used to signal availability or control access to a limited number of resources.

Why / significance: Use a mutex when the core need is exclusive ownership of a critical section; use a counting semaphore when multiple units of a resource may be available or when signaling is the main need.

Example / recall: Example: mutex for protecting a shared map; semaphore count 5 for a pool with five available connections.

6. What is deadlock?

Definition
Deadlock is a state where a set of processes or threads are permanently blocked because each is waiting for a resource or event that another member of the set cannot release.

Why / significance: It prevents progress until the system changes the resource situation or takes recovery action.

Example / recall: Example: Thread A holds lock 1 and waits for lock 2 while Thread B holds lock 2 and waits for lock 1.

7. Four conditions for deadlock?

Definition
The four necessary conditions are mutual exclusion, hold and wait, no preemption, and circular wait.

Why / significance: All four must hold simultaneously for a classic resource deadlock to be possible.

Example / recall: Memory cue: exclusive resource + hold one while waiting + cannot take it back + circular dependency.

8. What is virtual memory?

Definition
Virtual memory is a memory-management abstraction in which each process gets its own virtual address space, with virtual addresses translated to physical memory by hardware/OS-managed mappings.

Why / significance: It improves isolation and lets the OS manage physical memory independently of the address spaces seen by applications.

Example / recall: Paging is a common mechanism: virtual pages are mapped to physical frames, and non-resident pages can be brought in when needed.

9. What is a page fault?

Definition
A page fault occurs when a memory access refers to a virtual page that is not currently present in the required physical-memory mapping. The OS handles the fault, may load the page from secondary storage, updates the page table, and retries the access.

Why / significance: Page faults are normal in virtual-memory systems, but frequent faults can severely hurt performance because storage access is much slower than RAM.

Example / recall: Example: accessing a page that was evicted earlier can trigger a page fault and page-in operation.

10. Stack vs heap?

Definition
The stack is used for call frames and short-lived execution state associated with function calls. The heap is used for dynamically allocated objects/data whose lifetime is managed separately from a single call frame.

Why / significance: Stack allocation/access is typically fast and structured but limited; heap allocation is more flexible but involves allocator/runtime management and may have fragmentation or garbage-collection costs depending on the

environment.

Example / recall: Example: a function's local variables typically live in its stack frame, while a dynamically created object may live on the heap.

8. RAPID RECALL SHEET

Use these as prompts. Cover the right side and reconstruct the answer aloud.

Process	Program in execution + own address space/resources
Thread	Execution unit inside process + shared process resources
Context switch	Save one context -> restore another
Race condition	Result depends on concurrent timing/interleaving
Critical section	Shared-state code that needs synchronized access
Mutex	One owner at a time
Semaphore	Counter-based signaling/resource control
Deadlock	Permanent waiting cycle
4 deadlock conditions	Mutual exclusion + hold/wait + no preemption + circular wait
Virtual memory	Per-process virtual address space + address translation
Page fault	Needed page not currently resident/mapped as required
Paging	Pages in virtual memory + frames in physical memory
Stack vs heap	Call-frame/automatic state vs dynamic allocation

Spaced-repetition use
1. Read the question only.
2. Answer from memory in 30-60 seconds.
3. Check the canonical answer.
4. Fix only what was missing.
5. Re-test later: next day, 3 days later, 7 days later, then weekly until stable.

9. HIGH-YIELD DISTINCTIONS

Process vs thread	Process -> separate address space/isolation; thread -> shares process address space/resources.
Mutex vs semaphore	Mutex -> exclusive ownership; semaphore -> counter-based synchronization/resource control.
Prevention vs avoidance vs detection	Prevention -> break a necessary condition; avoidance -> keep state safe; detection -> find deadlock after it occurs.
Paging vs virtual memory	Paging -> fixed-size pages/frames and mapping; virtual memory -> broader per-process address-space abstraction.
Page fault vs context switch	Page fault -> memory-access exception/handling; context switch -> change of CPU execution context.
Stack vs heap	Stack -> call frames/automatic execution state; heap -> dynamic allocation/lifetime-managed data.

Note on scope: These notes follow the Operating Systems headings and ten interview questions in the supplied Infosys preparation roadmap. The explanations expand those headings for study use while preserving the roadmap terminology.